COURSE REPORT

ON

SQL Course

(Master the Fundamentals and Advanced Concepts DBMS)

Submitted by

VIBHANSHU MITTAL (Reg No.RA2411030030062)

Under the guidance of

Dr. Anirudha Gaikwad

(Assistant Professor, Department of Computer Science & Engineering)

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

of

FACULTY OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE & TECHNOLOGY,

DELHI-NCR CAMPUS

April 2026

(Under Section 3 of UGC Act, 1056)

BONAFIDE CERTIFICATE

BONAFIDE CERTIFICATE

This is to certify that the course report entitled " **Master the Fundamentals and Advanced Concepts**" submitted by **VIBHANSHU MITTAL (Reg No.RA2411030030062)** of **B.Tech — Computer Science and Engineering with specialization in Cyber Security**, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering**, is a record of the original work carried out by them under my supervision and guidance.
This project has been carried out during the academic year **2025–2026** at **SRM Institute of Science and Technology, Delhi-NCR Campus**, and represents the sincere efforts of the students. To the best of my knowledge and belief, this project has not been submitted, in part or full, for the award of any other degree or diploma at this institute or any other educational institution.

The project report embodies the results of investigations conducted by the students and demonstrates their understanding of the subject matter, their ability to carry out systematic research, and their capacity for critical analysis and independent thinking.

I am confident that the knowledge and experience gained by the students through this project will prove valuable for their academic and professional growth in the field of computer science and software development.

I further certify that all necessary ethical guidelines and academic standards have been followed in the preparation of this project report.

SIGNATURE

(Dr. Anirudha Gaikwad)
GUIDE
Assistant Professor, Dept. of CSE
SRM Institute of Science & Technology

SIGNATURE

(Dr. Neelam Singh)
HEAD OF DEPARTMENT (CSE)
Dept. of CSE
SRM Institute of Science & Technology

ACKNOWLEDGEMENT

I take this opportunity to express my profound gratitude and deep regards to all those who have extended their invaluable assistance and support in the successful completion of my project titled **"Master the Fundamentals and Advanced Concepts course"**.

First and foremost, I am deeply indebted to my esteemed guide, **Dr. Anirudha Gaikwad**, Assistant Professor, Department of Computer Science and Engineering, SRM Institute of Science and Technology, Delhi-NCR Campus, for his constant supervision, insightful guidance, and constructive feedback throughout the course of this project. His patience, motivation, and immense knowledge have been pivotal in shaping this work into its final form.

I am equally grateful to **Dr. Neelam Singh**, Head of Department, CSE for her encouragement, academic support, and for providing me with the opportunity and resources to undertake this project successfully.

I extend my sincere thanks to the faculty members and technical staff of the **Department of Computer Science and Engineering**, SRM Institute of Science and Technology, for their cooperation and assistance, which contributed immensely to the smooth execution of my project work.

I would also like to express my heartfelt appreciation to my **parents** and **friends**, whose unwavering support and encouragement kept me motivated throughout this journey. Their belief in my abilities gave me the strength to face challenges and accomplish my goals.

Lastly, I extend my gratitude to all the contributors of online resources and learning platforms such as **GeeksforGeeks**, **W3Schools**, **ChatGPT** and other references that enhanced my understanding and helped me overcome technical difficulties during project development.

This project has been a rewarding learning experience, enabling me to enhance my technical skills, deepen my knowledge of Database Management System and algorithmic thinking, and appreciate the importance of systematic design. I am truly thankful to all who, directly or indirectly, helped make this project a success.

I am truly thankful to all who, directly or indirectly, helped me make this project a success.

ACADEMIC COURSE COMPLETION REPORT

Database Management Systems (DBMS)

Course Title	DBMS Course – Master the Fundamentals and Advanced Concepts
Platform	Scaler Topics (by InterviewBit Technologies Pvt. Ltd.)
Instructor	Srikanth Varma, Lead DSML Instructor, Scaler
Duration	1 Day 5 Hours 17 Minutes | 16 Modules | 90 Lessons
Level	Beginner to Intermediate
Certificate	Certificate of Excellence – Scaler Topics
Completion Year	2026

Submitted in partial fulfilment of coursework requirements
B.Tech Program — Computer Science & Engineering

April 2026

Table of Contents

1. Introduction 3
2. Course Overview & DBMS Concepts Covered 4
3. Methodology 6
4. Analysis and Findings 8
5. Discussion & Recommendations 11
6. Conclusion 13

References 14

Appendices 15

1. Introduction

1.1 Background and Context

In an era defined by data-intensive computing, the ability to model, store, retrieve, and manage structured information forms the backbone of virtually every software system. Database Management Systems (DBMS) serve as the foundational infrastructure upon which enterprise applications, government platforms, e-commerce systems, and mission-critical services are built. For a student pursuing a career in software engineering, understanding DBMS is not optional — it is prerequisite knowledge that directly influences the quality of system design decisions.

This report documents the successful completion of the DBMS Course — Master the Fundamentals and Advanced Concepts, offered by Scaler Topics. The course was designed as a comprehensive progression from foundational database theory to advanced concepts including transaction management, concurrency control, and indexing strategies. It was undertaken with the specific objective of strengthening the theoretical grounding required for designing robust backend systems.

1.2 Motivation for Undertaking the Course

The motivation for enrolling in this course arose from a real-world project requirement — the development of a Military Inventory Management System. This system demanded a thorough understanding of relational data modelling, transactional integrity, concurrent access management, and efficient data retrieval strategies. The course provided the formal conceptual framework required to architect such a system responsibly and effectively.

1.3 Scope of this Report

This report presents a structured academic analysis of the course content, learning outcomes, the methodology employed during self-directed study, and the application of acquired knowledge in the capstone project. Section 2 surveys the course modules and core DBMS concepts. Section 3 describes the learning methodology and project development approach. Section 4 analyses findings with specific reference to the project. Section 5 presents a critical discussion and recommendations, and Section 6 draws concluding observations.

2. Course Overview & DBMS Concepts Covered

2.1 Course Profile

The Scaler Topics DBMS course is a structured, self-paced programme authored by Srikanth Varma, Lead DSML Instructor at Scaler and former Senior ML Scientist at Amazon. The course comprises 16 modules spanning 90 lessons, totalling approximately 29 hours of instructional content. A Certificate of Excellence is awarded upon successful completion.

Attribute	Detail
Total Modules	16
Total Lessons	90
Duration	1d 5h 17m (~29 hours)
Delivery Mode	Online, Self-Paced, Lifetime Access
Certification	Certificate of Excellence, Scaler Topics
Language	English (Audio & Subtitles)

2.2 Core Topics and Conceptual Coverage

2.2.1 Introduction to DBMS

The course commenced with an introduction to the purpose and architecture of a database management system, distinguishing DBMS from traditional file-based storage. Topics included the three-tier architecture (external, conceptual, and internal schemas), data independence, and the role of the database administrator. This foundational module established the vocabulary and mental models necessary for subsequent, more technical content.

2.2.2 Entity-Relationship (ER) Model and ER Diagrams

The ER model was studied as the primary tool for conceptual database design. The module covered entities, attributes (simple, composite, multivalued, derived), relationships, cardinality constraints (one-to-one, one-to-many, many-to-many), and participation constraints (total vs. partial). Students were also introduced to Extended ER features including generalisation, specialisation, and aggregation.

In the Military Inventory Management System, ER modelling was directly applied to identify entities such as UNIT, EQUIPMENT, PERSONNEL, REQUEST, TRANSFER, and AUDIT_LOG, and to define their inter-entity relationships — for example, a UNIT may submit multiple REQUESTs, each REQUEST is associated with a specific EQUIPMENT item, and each TRANSFER involves two UNITs.

2.2.3 Relational Model

The relational model module formalised the transition from ER diagrams to relational schemas. Key topics included the concept of a relation as a mathematical set, attributes and domains, tuples, keys (super key, candidate key, primary key, foreign key), relational algebra operations (select, project, join, union, difference, Cartesian product), and relational calculus. The formal foundation of the relational model ensures data integrity across a schema.

2.2.4 Structured Query Language (SQL)

SQL was covered across three functional categories: Data Definition Language (DDL) for schema creation and modification (CREATE, ALTER, DROP); Data Manipulation Language (DML) for data insertion, retrieval, update, and deletion (INSERT, SELECT, UPDATE, DELETE); and advanced querying constructs including various join types (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS JOIN), subqueries, aggregation functions, GROUP BY, HAVING, and view definitions.

The project employed PostgreSQL as the relational database engine, with Prisma ORM managing schema definitions and query execution. The SQL knowledge gained allowed for effective schema design, JOIN-heavy reporting queries (e.g., joining EQUIPMENT, UNIT, and TRANSFER tables), and transaction scripting.

2.2.5 Normalisation (1NF, 2NF, 3NF)

Normalisation theory was taught as the systematic process for eliminating data redundancy and ensuring update anomalies are avoided. The module covered functional dependency, Armstrong's axioms, the computation of attribute closures, and the progressive normal forms: First Normal Form (1NF) — atomicity of attributes; Second Normal Form (2NF) — elimination of partial dependencies; and Third Normal Form (3NF) — elimination of transitive dependencies. BoyceCodd Normal Form (BCNF) was also introduced as a stronger variant of 3NF.

Every table in the Military Inventory Management System was designed to satisfy at least 3NF, ensuring that each non-key attribute is dependent only on the primary key and on nothing else.

2.2.6 Transactions and ACID Properties

The module on transactions addressed the concept of a logical unit of work that must be executed atomically. The four ACID properties were examined in depth: Atomicity (all-or-nothing execution), Consistency (database transitions between valid states), Isolation (concurrent transactions do not interfere), and Durability (committed changes persist across failures). The

module also covered the transaction state diagram (active, partially committed, committed, failed, aborted) and the role of the transaction manager.

2.2.7 Indexing

Indexing was studied as the mechanism for efficient data retrieval without full table scans. Topics included dense vs. sparse indexes, primary (clustered) and secondary (non-clustered) indexes, multi-level indexes, and B-tree and B+-tree structures. The performance implications of index selection — particularly on read-heavy vs. write-heavy workloads — were discussed in the context of query optimisation.

2.2.8 Concurrency Control

The final major topic addressed the challenges of executing multiple transactions simultaneously. The module covered serializability theory (conflict and view serializability), precedence graphs, lock-based protocols (two-phase locking: growing and shrinking phases), timestamp-based protocols, and deadlock handling strategies including detection, prevention, and avoidance.

3. Methodology

3.1 Learning Approach

The course was undertaken following a structured self-directed learning methodology. Given its self-paced format, a weekly schedule was established to ensure systematic progression through all 16 modules. Each module was studied over a dedicated session that included: (i) watching video lectures in full, (ii) reviewing supplementary Scaler Topics articles on core concepts, (iii) solving the associated challenge problems, and (iv) relating each concept to the practical project in a personal knowledge log.

3.2 Supplementary Resources

To complement the course lectures, the following academic and professional resources were consulted throughout the learning period:

- Database System Concepts (Silberschatz, Korth, and Sudarshan, 7th Edition) — for formal definitions and proof-based treatment of relational algebra and transaction theory.
- PostgreSQL Official Documentation — for understanding platform-specific SQL syntax and indexing behaviour.
- Prisma ORM Documentation — for schema definition language (SDL), migration workflows, and query builder API.
- Next.js Documentation — for understanding server-side rendering patterns and API route integration with database layers.
- Scaler Topics DBMS Tutorial articles — specifically the Normalisation in DBMS and ACID Properties in DBMS pages, referenced directly in the course material.

3.3 Project Development Approach

The Military Inventory Management System was developed using an iterative, database-first development approach. The methodology followed four phases:

Phase 1: Conceptual Modelling

ER diagrams were constructed to identify all entities, attributes, and relationships. Cardinality constraints were mapped for all entity pairs. The conceptual model was reviewed for completeness and correctness before proceeding.

Phase 2: Logical Design

ER diagrams were converted to relational schemas. Normalisation analysis was conducted on each table to ensure compliance with 3NF. Foreign key relationships were defined, and constraints (NOT NULL, UNIQUE, CHECK) were specified.

Phase 3: Physical Implementation

The Prisma schema file was authored to define all models, fields, relations, and indexes. PostgreSQL was provisioned as the target database. Prisma Migrate was used to apply schema migrations in a reproducible, version-controlled manner. Database seed scripts were written to populate reference data.

Phase 4: Application Integration

Next.js API routes were implemented to expose CRUD operations, workflow endpoints (request submission, approval, transfer initiation), and reporting endpoints. All database interactions were routed through Prisma's type-safe query builder. Transaction blocks were employed for multi-step operations to enforce ACID guarantees.

3.4 Evaluation Criteria

The effectiveness of the course was evaluated along two dimensions: (i) performance on the 16 embedded challenge problems distributed across course modules, and (ii) the successful implementation of each DBMS concept in the capstone project. A concept was considered applied when it was demonstrably present in either the Prisma schema, a SQL query, or application-layer logic.

4. Analysis and Findings

4.1 Conceptual Mastery Analysis

Upon completing all 16 modules, the following analysis summarises the mastery level achieved for each major topic and its application in the Military Inventory Management System:

Topic	Key Concepts Mastered	Project Application
Introduction to DBMS	3-tier architecture, data independence, DBMS roles	Informed separation of data layer (PostgreSQL) from application layer (Next.js)
ER Modelling	Entities, attributes, cardinality, participation	Full ER diagram for 6+ entities with M:N relationships
Relational Model	Keys, foreign keys, relational algebra	Schema with FK constraints, join-based reporting
SQL	DDL, DML, joins, aggregation, subqueries	Prisma schema as DDL, complex join queries for audit reports
Normalisation	1NF, 2NF, 3NF, functional dependencies	All tables verified to 3NF; no update anomalies present
Transactions & ACID	ACID properties, transaction states, rollback	Prisma $transaction for approval and transfer workflows
Indexing	B+-tree, dense/sparse, clustered/nonclustered	Indexes on unitId, equipmentId, status fields for fast lookup
Concurrency Control	2PL, serialisability, deadlock handling	PostgreSQL MVCC; row-level locking for approval operations

4.2 Project Findings: Military Inventory Management System

4.2.1 System Overview

The Military Inventory Management System is a full-stack web application engineered to provide centralised, auditable management of military assets distributed across multiple organisational units. The system was developed using PostgreSQL (relational database), Prisma ORM (data access layer), and Next.js (application framework).

4.2.2 Database Schema and Normalisation

The relational schema comprised six primary tables: UNIT (representing organisational units), EQUIPMENT_ITEM (individual inventory assets), REQUEST (formal item requests raised by units), APPROVAL (authorisation records), TRANSFER_LOG (records of inter-unit equipment movement), and AUDIT_LOG (timestamped system event records). Every table was confirmed to be in Third Normal Form. For example, the EQUIPMENT_ITEM table stores a foreign key reference to UNIT rather than denormalising unit information into the equipment row — eliminating update anomalies that would otherwise arise when a unit is renamed.

4.2.3 Transaction Management

The equipment transfer workflow exemplifies the direct application of ACID transaction theory. A transfer operation modifies the EQUIPMENT_ITEM.currentUnitId, inserts a new TRANSFER_LOG record, and inserts an AUDIT_LOG entry — three distinct write operations. Using Prisma's $transaction API, these three operations are wrapped in a single atomic transaction. Should any operation fail (for instance, a constraint violation), the entire transaction is rolled back, preserving consistency.

Similarly, the request approval workflow — which updates the REQUEST.status field and triggers inventory allocation — is executed within a transaction to prevent partial updates that would leave the database in an inconsistent state.

4.2.4 Indexing Strategy

Query performance analysis during development identified several hot-path queries: retrieving all equipment for a given unit, listing pending requests by status, and fetching audit logs for a specified equipment item. Composite and single-column indexes were defined in the Prisma schema on the following fields: EQUIPMENT_ITEM.unitId, REQUEST.status, REQUEST.unitId, and AUDIT_LOG.equipmentId. These indexes correspond directly to the course coverage of nonclustered secondary indexes on frequently queried non-key attributes.

4.2.5 Audit Logging

The AUDIT_LOG table was designed as an append-only table to maintain an immutable history of all state-changing operations. This design pattern is directly informed by the DBMS concept of a system log (write-ahead log), which records transaction events for recovery purposes. Each audit entry captures the operation type, the affected entity identifier, the performing unit, and a timestamp — enabling full traceability of every inventory movement.

4.2.6 Concurrency Considerations

The project operates in a multi-user environment where multiple officers from different units may simultaneously submit requests, initiate transfers, or query inventory. PostgreSQL's Multi-Version Concurrency Control (MVCC) mechanism, which implements a form of snapshot isolation, was relied upon to ensure that concurrent read operations do not block write operations. For critical approval operations, SELECT ... FOR UPDATE was considered to implement row-level

pessimistic locking — a direct application of two-phase locking principles studied in the concurrency control module.

5. Discussion & Recommendations

5.1 Strengths of the Course

The Scaler Topics DBMS course demonstrated several pedagogical strengths that contributed to effective learning outcomes:

- Progressive conceptual layering: The course was sequenced such that each module built logically upon the preceding one. ER modelling preceded the relational model, which preceded SQL, which preceded normalisation — mirroring the natural workflow of database design.
- Theory-to-practice alignment: The inclusion of 16 challenge problems, one per module, ensured that each theoretical concept was immediately tested in application. This prevented passive consumption of content and reinforced retention.
- Conciseness without superficiality: At approximately 29 hours of content, the course covered both foundational and advanced topics (indexing, concurrency control) without requiring a semester-long commitment — making it highly suitable as a supplementary learning resource alongside formal coursework.
- Credentialed certification: The Certificate of Excellence issued by Scaler carries professional recognition, having been built by a faculty member with a 5-star rating from over 2,000 students and a distinguished academic background from IISc Bangalore.

5.2 Limitations Observed

While the course is comprehensive for its stated level (beginner to intermediate), several areas were identified where depth could be extended:

- Query Optimisation: The course introduced indexing but did not deeply cover query execution plans, cost-based optimisation, or the use of EXPLAIN/EXPLAIN ANALYZE in PostgreSQL. A learner aiming for database administration roles would benefit from supplementary study in this area.
- Distributed Databases: Topics such as data sharding, replication, and the CAP theorem, which are increasingly relevant in cloud-native systems, were not included. These represent a natural extension of the current curriculum.
- NoSQL and Polyglot Persistence: Modern software systems frequently combine relational and non-relational stores. The course appropriately focuses on relational DBMS, but a module on when and why to deviate from the relational model would enhance practical applicability.
- Hands-on SQL Environment: While an IMDB dataset link was provided, an embedded SQL sandbox within the platform would have reduced friction in experimenting with queries.

5.3 Recommendations for Future Learners

Based on the experience of completing this course, the following recommendations are offered to peers undertaking the same programme:

- Build a project concurrently: The most significant learning acceleration occurred when concepts were immediately applied in the Military Inventory Management System. Even a small side project greatly reinforces abstract material.
- Draw ER diagrams before writing code: This practice, instilled in Module 2 of the course, proved invaluable in preventing structural refactoring in later development stages. Conceptual correctness at the design phase translates to fewer migration issues at the implementation phase.
- Read the PostgreSQL documentation alongside the course: The course teaches vendorneutral DBMS theory; reading the PostgreSQL docs in parallel contextualises those concepts in the specific dialect and feature set of the target database.
- Revisit normalisation critically: 3NF is the standard target, but learners should understand scenarios where intentional denormalisation is justified for performance reasons — a nuance that the course introduces but does not extensively explore.

5.4 Application to Professional Practice

The combined theoretical and practical learning from this course has direct applicability to professional software engineering practice. Specifically: the ability to model data using ER diagrams reduces the likelihood of schema redesigns mid-project; knowledge of normalisation reduces the occurrence of data inconsistencies in production databases; understanding ACID and transaction management enables the design of workflows that are safe under failure conditions; and familiarity with indexing strategies allows developers to proactively address performance bottlenecks rather than reactively debugging slow queries.

6. Conclusion

This report has documented the completion of the Scaler Topics DBMS Course — Master the Fundamentals and Advanced Concepts, and presented a detailed academic analysis of the learning journey, methodological approach, and practical application of course content.

The eight core topic areas — Introduction to DBMS, ER Modelling, Relational Model, SQL, Normalisation, Transactions and ACID Properties, Indexing, and Concurrency Control — were studied systematically over 16 modules and 90 lessons. Each topic was not merely consumed academically but was deliberately applied in the development of the Military Inventory Management System, a full-stack application built with PostgreSQL, Prisma ORM, and Next.js.

The project served as both a validation of theoretical understanding and a demonstration of the practical value of formal database education. The normalised relational schema eliminated update anomalies; ACID-compliant transaction wrappers ensured the integrity of multi-step workflows; and strategic indexing improved query performance on high-frequency data access paths. The concurrency considerations addressed in the design reflect an awareness of production-grade requirements that extends beyond simple CRUD operations.

The course successfully bridged the gap between database theory and implementation practice. While certain advanced topics — particularly distributed data management and query optimisation — remain areas for further study, the foundational and intermediate competencies acquired are immediately applicable to backend system design, technical interviews, and professional database work.

In conclusion, the DBMS course from Scaler Topics represents a high-quality, time-efficient pathway to acquiring robust database knowledge. The skills developed through this course are foundational to every software system that relies on persistent, structured data — which is to say, nearly every system of consequence.

References

[1] Silberschatz, A., Korth, H. F., and Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.

[2] Ramakrishnan, R. and Gehrke, J. (2002). Database Management Systems (3rd ed.). McGraw-Hill.

[3] Scaler Topics. (2026). DBMS Course — Master the Fundamentals and Advanced Concepts. Retrieved from https://www.scaler.com/topics/course/dbms/

[4] Scaler Topics. (2026). Normalisation in DBMS. Retrieved from https://www.scaler.com/topics/dbms/normalization-in-dbms/

[5] Scaler Topics. (2026). ACID Properties in DBMS. Retrieved from https://www.scaler.com/topics/dbms/acid-properties-in-dbms/

[6] PostgreSQL Global Development Group. (2026). PostgreSQL 16 Documentation. Retrieved from https://www.postgresql.org/docs/

[7] Prisma. (2026). Prisma ORM Documentation. Retrieved from https://www.prisma.io/docs/

[8] Vercel. (2026). Next.js Documentation. Retrieved from https://nextjs.org/docs

[9] Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6), 377-387.

[10] Garcia-Molina, H., Ullman, J. D., and Widom, J. (2008). Database Systems: The Complete Book (2nd ed.). Prentice Hall.

Appendices

Appendix A: Course Module Breakdown

Module	Topic Area	Coverage
1	Introduction to DBMS	Architecture, Roles, Data Independence
2	ER Model and ER Diagrams	Entities, Relationships, Cardinality
3	Extended ER Features	Generalisation, Specialisation, Aggregation
4	Relational Model	Keys, Relational Algebra
5-6	SQL – DDL and DML	CREATE, ALTER, INSERT, SELECT, UPDATE, DELETE
7	SQL – Joins and Subqueries	INNER, OUTER, CROSS JOIN, Nested Queries
8-9	Normalisation (1NF-3NF/BCNF)	Functional Dependencies, Normal Forms
10	Transactions and ACID	Transaction States, ACID Properties
11	Indexing	B+-Tree, Clustered vs. NonClustered
12-13	Concurrency Control	2PL, Serializability, Deadlock Handling
14-16	Advanced Topics & Revision	Query Optimisation Overview, Module Recap

Appendix B: Technology Stack — Military Inventory Management System

Layer	Technology	Role
Database	PostgreSQL	Relational DBMS — persistent data storage, ACID, indexing
ORM / Data Access	Prisma ORM	Type-safe schema definition, migrations, query builder
Application Framework	Next.js	Full-stack React framework with API routes for backend logic
Layer	**Technology**	**Role**
Transaction Handling	Prisma $transaction	Atomic multi-operation blocks for transfer and approval workflows
Concurrency	PostgreSQL MVCC	Snapshot isolation for concurrent multi-user access

Appendix C: DBMS Concepts Mapped to Project Features

DBMS Concept	Project Implementation
ER Modelling	Full ER diagram with UNIT, EQUIPMENT, REQUEST, APPROVAL, TRANSFER_LOG, AUDIT_LOG entities
3NF Normalisation	All tables designed to 3NF; no partial or transitive dependencies present
Foreign Key Constraints	Referential integrity enforced via Prisma @relation directives
ACID Transactions	Transfer and approval workflows wrapped in $transaction for atomicity
Indexing	Indexes on unitId, status, and equipmentId for query performance
Audit Logging	Append-only AUDIT_LOG modelled on write-ahead log principles
Concurrency Control	PostgreSQL MVCC for snapshot isolation; row-level locking for critical writes

Appendix D: Glossary of Key Terms

ACID: Atomicity, Consistency, Isolation, Durability — the four properties guaranteeing reliable transaction execution.

B+-Tree: A balanced tree data structure used in database indexing for efficient range and equality queries.

DDL: Data Definition Language — SQL commands for defining and modifying database schema (CREATE, ALTER, DROP).

DML: Data Manipulation Language — SQL commands for data operations (INSERT, SELECT, UPDATE, DELETE).

ER Model: Entity-Relationship Model — a conceptual schema design technique for modelling data entities and their relationships.

MVCC: Multi-Version Concurrency Control — a PostgreSQL concurrency mechanism allowing readers and writers to operate without blocking each other.

Normalisation: The process of organising relational database columns and tables to reduce data redundancy and improve data integrity.

ORM: Object-Relational Mapper — a software layer that maps object-oriented code structures to relational database tables.

Serializability: A correctness criterion for concurrent transaction schedules, equivalent in effect to serial execution.

Two-Phase Locking (2PL): A concurrency control protocol where transactions acquire all locks in a growing phase and release them in a shrinking phase.